

# Cost Optimization

Savings Plans · Right-Sizing · Tagging · Data Transfer · FinOps Culture · ShopFlow

AWS Tutorial · ShopFlow

## 19-01

## Why Cloud Cost Optimization Matters for ShopFlow

ShopFlow starts on AWS spending \$3,000/month. After 18 months of rapid growth without cost governance, the bill is \$47,000/month. Revenue grew 5x but cloud costs grew 15x. This chapter teaches you to bend the curve.

| ShopFlow Spending Growth | Month 1 | Month 6  | Month 18 | After Optimization |
|--------------------------|---------|----------|----------|--------------------|
| ECS Fargate (compute)    | \$320   | \$2,100  | \$18,400 | \$9,200 (-50%)     |
| RDS Aurora (database)    | \$180   | \$890    | \$5,600  | \$3,100 (-45%)     |
| Data transfer            | \$45    | \$410    | \$4,200  | \$1,800 (-57%)     |
| ElastiCache Redis        | \$60    | \$280    | \$1,800  | \$980 (-46%)       |
| S3 + CloudFront          | \$30    | \$190    | \$2,900  | \$1,200 (-59%)     |
| Total                    | \$3,000 | \$11,200 | \$47,000 | \$22,400 (-52%)    |

### warn The Optimization Mindset

Cost optimization is not about being cheap — it is about getting the same outcome for less money. ShopFlow can serve 1M MAU for \$22,400/month or for \$47,000/month. The difference is \$296,400/year that goes toward engineering salaries, marketing, or profit. Cloud cost optimization is a competitive advantage.

## 19-02

## AWS Cost Management Tools

AWS provides four tools for understanding and controlling costs. Use all four — they serve different purposes.

| Tool                            | Purpose                                                                                                                      | How ShopFlow Uses It                                                                   |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| AWS Cost Explorer               | Visualize spending by service, account, region, and time period. Identify top services and highest, month-over-month growth. | Review growth over time. Identify top 5 services by cost.                              |
| AWS Budgets                     | Set spending thresholds and receive email/SMS alerts when the actual budget is exceeded.                                     | \$50K/month actual budget with a 20% (\$40K) limit.                                    |
| AWS Cost and Usage Report (CUR) | Hourly/daily CSV export of every AWS resource.                                                                               | Exported daily to S3. Allowed to query data. Quick look at the most significant costs. |
| AWS Compute Optimizer           | Machine learning recommendations for right-sizing EC2, Lambda, ECS, and Fargate.                                             | Right-sizing EC2. Adopt ECS and Fargate. No wasted capacity.                           |

## 19-03

## Cost Allocation Tags — Know Who Spends What

Without tags, your AWS bill is one big number. With tags, you know that the checkout microservice costs \$3,200/month and the search service costs \$8,400/month. Tags enable accountability.

## Activate Cost Allocation Tags

### ■ Required Tags for ShopFlow

■ Tags must be activated in Cost Explorer before they appear in cost reports (24-hour delay after first tagging).

Tag Key 1

Environment (values: production, staging, development)

Tag Key 2

Service (values: product-svc, order-svc, user-svc, search-svc, checkout-svc)

Tag Key 3

Team (values: platform, growth, commerce)

Tag Key 4

CostCenter (values: engineering, data, infrastructure)

■ AWS cost allocation tags are case-sensitive. Enforce with AWS Config rule: required-tags. Non-compliant resources (untagged) trigger SNS alert to team lead.

### Activate Tags

```
// CDK: enforce tags on all resources in a stack // Apply in your ShopFlow CDK app before
synthesizing import * as cdk from "aws-cdk-lib"; import { Aspects } from "aws-cdk-lib"; class
RequiredTagsAspect implements cdk.IAspect { visit(node: cdk.IConstruct): void { if (node
instanceof cdk.CfnResource) { cdk.Tags.of(node).add("Environment", process.env.ENVIRONMENT ||
"development"); cdk.Tags.of(node).add("Service", process.env.SERVICE_NAME || "unknown");
cdk.Tags.of(node).add("Team", process.env.TEAM_NAME || "unknown");
cdk.Tags.of(node).add("ManagedBy", "cdk"); } } } const app = new cdk.App(); const stack = new
ShopFlowProductStack(app, "ShopFlowProduct"); Aspects.of(stack).add(new RequiredTagsAspect());
// tags every resource in stack
```

19-04

## Savings Plans — Commit to Save 40-66%

On-demand pricing is the most expensive way to run AWS. If you know you will use a service for 1-3 years, Savings Plans let you commit to a spend level in exchange for significant discounts.

| Commitment Type                                    | Discount vs On-Demand | Flexibility                                                                                                    | ShopFlow Recommendation                               |
|----------------------------------------------------|-----------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Compute Savings Plans (1 year, up to 54% off)      | Up to 54% off         | Applies to any EC2 instance type, size, REGION, OS, and AWS CloudFormation Stack                               | Buy Compute Savings Plans                             |
| EC2 Instance Savings Plans (1 year, up to 72% off) | Up to 72% off         | Applies to a specific instance family (e.g. c5) in a specific region. Less flexible than Compute Savings Plans | ShopFlow recommends EC2 Instance Savings Plans        |
| RDS Reserved Instances (1 year, up to 40% off)     | Up to 40% off         | Specific DB engine, instance class, and REGION                                                                 | ShopFlow recommends RDS Reserved Instances            |
| ElastiCache Reserved Nodes (1 year, up to 43% off) | Up to 43% off         | Specific cache node type and region.                                                                           | Reserve cache.r6g.xlarge for production Redis cluster |

### ■ Savings Plan Strategy

The rule of thumb: analyze your last 3 months of usage. Find your "baseline" — the minimum compute you used every single hour. Buy Savings Plans to cover 70% of that baseline. Leave 30% uncovered as on-demand buffer for unexpected growth. Never over-commit: if you shut down a service, you still pay for the Savings Plan commitment.

Example: ShopFlow example: lowest-usage hour in the last 3 months cost \$18.40 in compute. 70% of \$18.40 = \$12.88/hour commitment. Annual Compute Savings Plan: \$12.88 x 8,760h = \$112,829 spend commitment saves ~52% = \$58,670/year.

## Right-Sizing ECS Fargate — CPU and Memory

ShopFlow originally launched with generous Fargate task sizes "to be safe." Six months later, actual utilization data shows most services are running at 15-25% CPU and 30-40% memory. Right-sizing recovers that money.

| Service          | Original Size | Avg CPU | Avg Memory | Recommended Size                        | Monthly Savings               |
|------------------|---------------|---------|------------|-----------------------------------------|-------------------------------|
| product-svc      | 2 vCPU / 4 GB | 18%     | 35%        | 1 vCPU / 2 GB (-50% CPU, -50% mem)      | \$680/month (6 tasks x \$113) |
| order-svc        | 2 vCPU / 4 GB | 22%     | 42%        | 1 vCPU / 3 GB (-50% CPU, -25% mem)      | \$510/month                   |
| search-svc       | 4 vCPU / 8 GB | 31%     | 55%        | 2 vCPU / 6 GB (-50% CPU, -25% mem)      | \$720/month                   |
| user-svc         | 1 vCPU / 2 GB | 12%     | 28%        | 0.5 vCPU / 1 GB (-50% CPU, -50% mem)    | \$240/month                   |
| notification-svc | 1 vCPU / 2 GB | 8%      | 22%        | 0.25 vCPU / 0.5 GB (-75% CPU, -75% mem) | \$350/month                   |

## tip How to Right-Size Safely

Never right-size in production without a phased approach: (1) Reduce task size by 25% in staging. Run load tests at 2x production traffic. (2) Deploy to 10% of production tasks (blue/green canary). Monitor p99 latency for 48 hours. (3) If latency is unchanged, roll out to 100%. (4) Keep the old task definition — rollback takes 2 minutes if needed.

## Aurora PostgreSQL Cost Optimization

Aurora costs money in four ways: instance hours, storage (per GB-month), I/O operations (per million), and data transfer. ShopFlow reduced its Aurora bill from \$5,600 to \$3,100/month by addressing all four.

| Cost Component                             | Original Cost | Optimization                                            | New Cost             |
|--------------------------------------------|---------------|---------------------------------------------------------|----------------------|
| Writer instance (r5.2xlarge, on-demand)    | \$1,680/month | Switch to 1-year Reserved Instance (no upfront)         | \$1,008/month (-40%) |
| Reader instances (2x r5.xlarge, on-demand) | \$2,680/month | Right-size to r5.large (peak CPU: 28%); reserve 1       | \$672/month (-60%)   |
| Aurora storage (4 TB)                      | \$480/month   | Enable Aurora Optimized Writes (reduces I/O for VLS)    | \$311/month (-35%)   |
| I/O operations (38B ops/month)             | \$760/month   | Add read replica to offload analytics queries; optimize | \$420/month (-45%)   |
| Backup storage (7-day retention)           | \$380/month   | Reduce automated backup window to 7 days (was 30 days)  | \$190/month (-50%)   |
| Snapshot exports (weekly to S3)            | \$168/month   | Keep weekly; delete snapshots > 90 days automatically   | \$93/month (-43%)    |

## S3 Storage Classes — Tiered Storage for the Data Lake

ShopFlow's S3 data lake grows by ~50 GB/day (Firehose + order archives + user uploads). Standard S3 at \$0.023/GB-month becomes expensive fast. Lifecycle policies automatically move data to cheaper storage tiers as it ages.

| Storage Class                      | Price/GB-month | Access Pattern                                  | ShopFlow Use                                   |
|------------------------------------|----------------|-------------------------------------------------|------------------------------------------------|
| S3 Standard                        | \$0.023        | Frequently accessed (read > 1/month)            | Last 30 days of Firehose event data (Athena c  |
| S3 Standard-IA (Infrequent Access) | \$0.0125       | Accessed < 1/month but must retrieve in 30-90   | 30-90 day event archive. Athena can still quer |
| S3 Glacier Instant Retrieval       | \$0.004        | Accessed quarterly, retrieval in ms             | 90-365 day archive. Product images > 1 year    |
| S3 Glacier Flexible Retrieval      | \$0.0036       | Accessed yearly, retrieval in 3-5 hours         | Order records > 1 year (compliance retention:  |
| S3 Glacier Deep Archive            | \$0.00099      | Accessed < once per year, retrieval in 12 hours | Logs and events > 3 years old. Pure complian   |

```
# CDK: S3 lifecycle policy for ShopFlow data lake
from aws_cdk import aws_s3 as s3, Duration
bucket = s3.Bucket(self, "DataLake", bucket_name="shopflow-analytics-lake", lifecycle_rules=[
    s3.LifecycleRule( id="tier-event-data", prefix="events/", transitions=[ s3.Transition(
        storage_class=s3.StorageClass.INFREQUENT_ACCESS, transition_after=Duration.days(30)), # after
    30 days s3.Transition( storage_class=s3.StorageClass.GLACIER_INSTANT_RETRIEVAL,
```

```

transition_after=Duration.days(90)), # after 90 days s3.Transition(
storage_class=s3.StorageClass.GLACIER, transition_after=Duration.days(365)), # after 1 year ],
expiration=Duration.days(2555) # delete after 7 years (compliance) ), s3.LifecycleRule(
id="product-images", prefix="product-images/", transitions=[ s3.Transition(
storage_class=s3.StorageClass.INFREQUENT_ACCESS, transition_after=Duration.days(90)), ] ), ])

```

19-08

## Data Transfer Costs — The Hidden Bill

Data transfer charges are the most misunderstood AWS cost. ShopFlow was paying \$4,200/month in data transfer fees. After analysis, 78% of that cost came from two fixable mistakes.

| Transfer Type                    | Cost                           | ShopFlow's Mistake                                                                                        | Fix                                                |
|----------------------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| EC2/ECS -> internet              | \$0.09/GB after first 1GB free | productsvc returning full product objects (5KB graphQL response fields when mobile clients need only 1KB) | Amazon CloudFront                                  |
| Cross-AZ data transfer           | \$0.01/GB each direction       | ECS tasks in AZ-1 calling Aurora reader in AZ-2 (Aurora reader endpoint has 150GB/s limit)                | Amazon ElastiCache                                 |
| ECS -> S3 (via internet gateway) | \$0.09/GB                      | Firehose delivering to S3 routed through CloudFront endpoint (S3 Gateway type, FREE)                      | Amazon S3                                          |
| EC2 -> CloudFront origin proxy   | \$0.085/GB                     | ALB serving product images directly; CDN cache hit rate < 23%                                             | Move images to S3+ CloudFront. Cache hit rate: 99% |

19-09

## Lambda Cost Optimization — Memory and Duration

Lambda pricing = (number of requests x \$0.00000002) + (GB-seconds x \$0.0000166625). The GB-seconds term is memory x duration. Counterintuitively, giving Lambda more memory often REDUCES cost because faster execution more than offsets the higher memory charge.

| Lambda Function               | Memory   | Avg Duration | GB-seconds/invoc | Monthly Cost (10M invoc)            |
|-------------------------------|----------|--------------|------------------|-------------------------------------|
| Order processor (original)    | 512 MB   | 2,400ms      | 1.228            | \$204                               |
| Order processor (power-tuned) | 1,792 MB | 680ms        | 1.225            | \$204 (same!) but 3.5x faster       |
| Trending engine (original)    | 256 MB   | 1,100ms      | 0.275            | \$46                                |
| Trending engine (power-tuned) | 512 MB   | 480ms        | 0.240            | \$40 (13% cheaper AND 2.3x faster)  |
| Fraud detector (original)     | 1,024 MB | 3,200ms      | 3.277            | \$546                               |
| Fraud detector (power-tuned)  | 3,008 MB | 890ms        | 2.671            | \$445 (18% cheaper AND 3.6x faster) |

### ■ AWS Lambda Power Tuning

AWS Lambda Power Tuning is an open-source Step Functions state machine that runs your Lambda at multiple memory configurations, measures duration and cost at each setting, and produces a recommendation. Run it against every Lambda function that executes more than 1 million times per month — the ROI is typically 10-30% cost reduction with a latency improvement as a bonus.

Example: Deploy: `aws cloudformation deploy --template-url https://s3.amazonaws.com/aws-lambda-power-tuning/sam/deploy.yml`. Then invoke the Step Function with your Lambda ARN and a payload sample. Takes 5 minutes to run.

19-10

## CloudFront Cost and Cache Optimization

CloudFront pricing has two main components: data transfer out to internet (\$0.0085-\$0.085/GB depending on region) and HTTP requests (\$0.01/10,000 requests). A higher cache hit rate reduces BOTH — fewer origin requests and less data transferred.

| Cache Hit Rate        | Origin Requests/month    | CloudFront Bill (10B req/month)              | Origin (ALB+ECS) bill impact               |
|-----------------------|--------------------------|----------------------------------------------|--------------------------------------------|
| 23% (ShopFlow before) | 7.7B requests hit origin | \$210 data transfer + \$100 requests = \$310 | 7.7B origin requests saturating ECS tasks; |
| 94% (ShopFlow after)  | 600M requests hit origin | \$85 data transfer + \$100 requests = \$185  | 600M origin requests; ECS has headroom;    |

| Cache Optimization Technique                                                                  | Before                                                    | After                                                                                   | How                                                                   |
|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Product images: set Cache-Control header (even on requests for static files)                  | Cache hit: 0% (even on requests for static files)         | Cache hit: 99% (browser and CDN cache for long time)                                    | Pages are for changing content-addressable URLs (content-addressable) |
| Product detail API: cache at CDN with 5-min TTL (API not cacheable)                           | Cache hit: 0% (API not cacheable)                         | Cache hit: 78% (most products are static)                                               | Add Cache-Control: public, max-age=300 to response                    |
| Category pages: ESI (Edge Side Include) for personalized content (Static framework cacheable) | Full page served from origin (Static framework cacheable) | Static framework cacheable; edge; personalized content fetched from CDN (1 hour). Fetch | Personalize page served from CDN (1 hour). Fetch                      |

19-11

## Hands-On Lab — Set Up AWS Budgets and Anomaly Detection

In this lab you will configure a monthly budget with alerts and enable Cost Anomaly Detection so AWS automatically notifies you of unexpected spending.

aws > AWS Budgets > Create budget

### Create Monthly Spend Budget

#### Budget Configuration

Budget type

Cost budget

Budget name

ShopFlow-Monthly-Total

Budget amount

\$50,000

Budget period

Monthly

#### Alert Thresholds

Alert 1 — Actual spend at 80%

\$40,000 — Email: devops@shopflow.com + SNS topic: shopflow-cost-alerts

Alert 2 — Actual spend at 95%

\$47,500 — Email: cto@shopflow.com, devops@shopflow.com + PagerDuty

Alert 3 — Forecasted spend at 100%

\$50,000 (forecasted) — Email: cto@shopflow.com — investigate before end of month

Create budget

## Create Anomaly Detection Monitor

### Monitor Configuration

Monitor type

AWS services (monitors all services for anomalies) ▀

Monitor name

ShopFlow-All-Services-Anomaly

### Alert Subscription

Alert threshold

Alert when anomaly impact > \$200 (absolute) OR > 20% (percentage) of expected daily spend

Notification email

devops@shopflow.com

Cost Anomaly Detection uses ML to establish your normal spending pattern. Alerts fire only for genuine anomalies, not normal daily variation. Typical response time: anomaly detected within 24 hours of occurrence.

Create monitor

19-12

Common Cost Waste Patterns — ShopFlow Audit Results

ShopFlow's first cost audit found 11 categories of waste totaling \$8,200/month. These are the most common waste patterns found in every growing startup on AWS.

| Waste Pattern            | ShopFlow Finding                                                                                                                                                                               | Monthly Waste | Fix |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-----|
| Idle resources           | 3 dev/staging ECS clusters running 24/7 (\$1,200 used 8h/day) — change business hours to 9am, auto-start at 8am using CloudFormation                                                           | \$1,200       |     |
| Orphaned resources       | 12 unused Elastic IPs (created during testing, never released) — Add --release-addresses --query 'Addresses[0].ReleaseOnDelete' to describe-elastic-ips --query 'Addresses[0].ReleaseOnDelete' | \$48          |     |
| Oversized NAT Gateways   | 3 NAT Gateways processing 8 TB/month (\$780) — Move S3 and DynamoDB traffic to VPC Gateway Endpoints                                                                                           | \$780         |     |
| Unused Snapshots         | 450 GB of EBS snapshots from terminated instances never deleted — Run weekly: find snapshots with no attached volumes and delete them                                                          | \$28          |     |
| CloudWatch Log retention | All CloudWatch log groups set to "Never expire" — \$63 TB stored at \$0.03/GB — Set retention for application logs, 7 days for operational logs                                                | \$63          |     |
| Unused Load Balancers    | 2 ALBs from deprecated microservices still running (\$0.005/hourly ALB cost) — Remove them                                                                                                     | \$70          |     |

19-13

Building a FinOps Culture at ShopFlow

Technology alone does not reduce cloud costs — culture does. ShopFlow's most effective cost reduction came not from automation but from making engineers aware of and accountable for their service's cost.

| Practice                      | What ShopFlow Does                                                                                                                                                                                        | Impact                                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Cost attribution by service   | Each microservice tagged. Weekly Slack digest: "Each microservice tagged. Weekly Slack digest: "Engineers proactively optimized services (e.g. moved dev/staging to Spot instances) — 12% cost reduction" | \$2,340 saved weekly (12% cost reduction)                                                                |
| Cost per unit metric          | Track "cost per order" and "cost per active user" alongside code commit metrics. Target: \$0.022/order. While order volume grew, cost per order decreased.                                                | Engineers optimized services to target \$0.022/order. While order volume grew, cost per order decreased. |
| Architecture review cost gate | New design reviews require cost estimate: "this design will cost \$X/month ML team will not build this design" — Rejected designs that would have cost \$10,000/month                                     | Engineers avoided \$10,000/month ML team will not build this design                                      |
| Monthly cost review           | 30-minute engineering all-hands: top 5 cost changes, team suggestions, optimization opportunities — Engineers identified \$1,500 in savings                                                               | Engineers identified \$1,500 in savings                                                                  |

Q

What is the difference between Savings Plan and Reserved Instances?

A

Savings Plan is a commitment to a certain dollar amount per hour spend level in exchange for discounts. Reserved Instance

How do you calculate the right Savings Plan commitment for Cost Explorer?

1. Export 6 months of Cost Explorer data. 2. Find the minimum hourly compute spend across all 2,160

What is a VPC Endpoint and how does it work?

A VPC Endpoint is a resource in your VPC that enables your resources in your VPC to communicate with AWS services (S3, DynamoDB, SQS)

How does ShopFlow reduce data transfer costs using VPC Endpoints?

Transfer techniques: (1) VPC Endpoints eliminate NAT Gateway charges for S3/DynamoDB traffic. (2)

What is AWS Cost Anomaly Detection and how does it work?

Cost Anomaly Detection uses ML to learn your normal daily spending pattern by service. It alerts when

AWS Compute Optimizer analyzes 14 days of CloudWatch metrics and produces machine-learning-based recommendations for EC2, ECS Fargate tasks, Lambda, and EBS volumes. ShopFlow runs it monthly.

aws > AWS Compute Optimizer > ECS on Fargate

## Fargate Task Right-Sizing Recommendations

### product-svc recommendation

Current task: 2 vCPU / 4096 MB | Risk: Over-provisioned

vCPU utilization (14-day p99)

22% (peak 38% during flash sale)

Memory utilization (14-day p99)

36%

Recommended task size: 1 vCPU / 2048 MB — projected savings \$680/month

### order-svc recommendation

Current task: 2 vCPU / 4096 MB | Risk: Over-provisioned

vCPU utilization (14-day p99)

24%

Memory utilization (14-day p99)

43%

Recommended task size: 1 vCPU / 3072 MB — projected savings \$510/month

Compute Optimizer requires Enhanced Monitoring enabled on ECS services. Enable in ECS cluster settings: Container Insights ON.

### tip Safe Right-Sizing Workflow

Before applying Compute Optimizer recommendations: (1) Check p99.9 utilization, not just p99 — the 0.1% worst case matters for latency. (2) Verify the observation window includes any recent traffic spikes (product launch, marketing campaign). (3) Apply in staging first with a load test at 2x production traffic. (4) Deploy 10% canary in production for 48 hours before full rollout.

Spot Instances use spare AWS capacity at up to 90% discount vs on-demand. The trade-off: AWS can reclaim the instance with a 2-minute warning. This makes Spot ideal for batch processing, CI/CD builds, and stateless workers — but NOT for ShopFlow's production API services.

| Workload                               | Spot Suitable?  | Reason                                                                                 | ShopFlow Savings                             |
|----------------------------------------|-----------------|----------------------------------------------------------------------------------------|----------------------------------------------|
| CodeBuild CI/CD build agents           | YES — excellent | Build jobs are short (5-15 min), stateless, and can be interrupted.                    | \$290/month (was \$410) on CodeBuild         |
| Flink analytics workers (non-critical) | YES — with care | Use Spot for additional Flink task managers in staging. Production staging/development | \$180/month. Provisioning dev infra          |
| Redshift (dev cluster)                 | YES             | Dev Redshift cluster runs batch queries during off-hours.                              | \$240 less (was \$480) on dev cluster        |
| ECS Fargate production API             | NO              | 2-minute interrupt notice means in-flight requests. Not applicable expenses.           | \$0.2k/month New                             |
| Aurora production database             | NO              | Databases require stable connections. Spot inapplicable.                               | Not applicable. In data section on migration |

19-17

DynamoDB Cost Optimization — On-Demand vs Provisioned

DynamoDB costs come from read/write request units, storage, backups, and data transfer. ShopFlow had three DynamoDB tables all set to on-demand mode — right for unpredictable workloads, wrong for stable ones.

| Table                                                  | Traffic Pattern                                                           | Recommended Mode                              | Monthly Cost Change                 |
|--------------------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------|-------------------------------------|
| shopflow-sessions (user sessions)                      | Spiky, low overnight, peaks 9am-10pm with sharp spikes in product launch  | On-Demand (keeps as is)                       | On-Demand. Savings on request units |
| shopflow-cart (shopping cart)                          | Highly correlated with checkout flow. Spikes during sales but predictable | On-Demand but set to auto scale (Provisioned) | \$180/month for 10 months           |
| shopflow-product-cache (trending item recommendations) | Very stable, predictable, trending every 30 days with less of a spike     | Provisioned with auto-scaling                 | \$180 less (was \$360) on prod DB   |

```
# CDK: DynamoDB provisioned with auto-scaling for product-cache table from aws_cdk import
aws_dynamodb as dynamodb, RemovalPolicy table = dynamodb.Table(self, "ProductCache",
table_name="shopflow-product-cache", partition_key=dynamodb.Attribute( name="metricKey",
type=dynamodb.AttributeType.STRING), billing_mode=dynamodb.BillingMode.PROVISIONED, # NOT
PAY_PER_REQUEST read_capacity=500, # baseline (will auto-scale up to 5000) write_capacity=100,
# baseline (will auto-scale up to 1000) removal_policy=RemovalPolicy.RETAIN) # Auto-scaling:
scale up when utilization > 70%, scale down when < 30% read_scaling =
table.auto_scale_read_capacity(min_capacity=500, max_capacity=5000)
read_scaling.scale_on_utilization(target_utilization_percent=70) write_scaling =
table.auto_scale_write_capacity(min_capacity=100, max_capacity=1000)
write_scaling.scale_on_utilization(target_utilization_percent=70)
```

19-18

Cost Explorer — Finding and Fixing Spending Anomalies

AWS Cost Explorer lets you slice your bill by 12 dimensions including service, region, usage type, instance type, and tags. ShopFlow's SRE lead does a structured 15-minute Cost Explorer review every Monday morning.

Weekly Cost Review — Monday Morning Checklist

Week-over-Week Comparison

Date range: Last 7 days vs prior 7 days. Granularity: Daily. Group by: Service.

Filter: Linked account

All accounts (production + staging + dev)

Red Flags to Investigate

Flag any service with week-over-week increase > 15% AND absolute increase > \$100

ECS Fargate this week

\$12,450 (was \$10,980 last week — +13.4%, +\$1,470) — Review ECS task count trend. Did we scale out and forget to scale in?

EC2-Other (data transfer) this week

\$1,890 (was \$980 last week — +92.9%, +\$910) — INVESTIGATE: nearly doubled. Check which service had traffic spike.

RDS this week

\$3,120 (was \$3,080 last week — +1.3%, +\$40) — Normal variance. No action needed.

Drill down: EC2-Other -> Usage Type -> DataTransfer-Out-Bytes by Service tag -> found: search-svc transferred 4.2 TB vs 2.1 TB prior week. Root cause: new product export feature added by growth team without review.

19-19

Multi-Account Strategy — AWS Organizations and Cost Governance

ShopFlow uses three AWS accounts: production, staging, and development. Managing costs across accounts requires AWS Organizations with consolidated billing and Service Control Policies (SCPs) to prevent accidental expensive resource creation in dev.

| Account              | Purpose                                                               | Cost Controls                                                                       |
|----------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Production (primary) | Live traffic: 1M MAU. All production infrastructure.                  | Budget: \$45K/month. SCP: deny EC2 instance types > r5.4xlarge without approval.    |
| Staging              | Load testing, QA, integration tests. Mirrors production architecture. | Budget: \$4K/month. EventBridge rule: shut down all ECS services at 8pm.            |
| Development          | Individual developer sandboxes. Ephemeral.                            | Budget: \$1.5K/month. SCP: Deny RDS instance creation. SCP: deny IAM role creation. |

info PR Environment Cost Control

Each GitHub PR creates a CDK-deployed dev environment (ECS tasks + RDS + ElastiCache). These cost ~\$18/day. A GitHub Action on PR close destroys the stack (cdk destroy). A Lambda runs daily and destroys any PR environment for a PR that has been closed for > 2 hours — prevents orphaned environments from forgotten branches costing \$500+/month.

19-20

Cost Dashboard — Engineering Visibility into Spending

The most powerful cost optimization tool at ShopFlow is a QuickSight dashboard where every engineer can see their service's daily cost. Visibility creates accountability without management overhead.

| Dashboard Widget                   | Data Source                          | What Engineers See                                                                            | Action Triggered                                      |
|------------------------------------|--------------------------------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Cost per service (daily bar chart) | QuickSight -> Athena (queried daily) | Each QuickSight SPICE cell, cost today, cost yesterday, cost 7 days prior, cost 30 days prior |                                                       |
| Cost per order (unit economics)    | QuickSight -> Athena (queried daily) | \$ per item today vs QuickSight (\$0.022) set trend over 90 days                              | Alerts 90 days > \$0.030 for 3 consecutive days       |
| Waste leaderboard                  | Lambda scans for idle resources      | \$ per day on idle resources by estimated cost                                                | Alerts if idle resources are found in the leaderboard |

| Dashboard Widget      | Data Source                 | What Engineers See                     | Action Triggered                       |
|-----------------------|-----------------------------|----------------------------------------|----------------------------------------|
| Savings Plan coverage | Cost Explorer API -> Lambda | % of code/site covered by Savings Plan | Alert if coverage drops to 70%/60%/50% |

19-21

Black Friday Cost Planning — Spend More on the Right Day

Black Friday is the one day ShopFlow INTENTIONALLY spends more. The goal is not to minimize cost on Black Friday — it is to ensure the infrastructure can handle 10x traffic without service degradation. But unplanned spending on surrounding days is waste.

| Period                        | Strategy                                                                      | Expected Cost                               | Key Decisions                                         |
|-------------------------------|-------------------------------------------------------------------------------|---------------------------------------------|-------------------------------------------------------|
| Nov 1-20 (pre-BF preparation) | Load testing with Spot instances. CDK + S3 for config                         | \$800 cost                                  | Early buildout test 152k file (90% disk space)        |
| Nov 21-26 (BF week)           | Scale out 5 days before: ECS desired capacity, ELB, S3, CloudFront, IAM, etc. | \$8,500, 10k/s is shared                    | 2x, 220k/s (2x) diff. from normal                     |
| Nov 27 (Black Friday)         | Maximum capacity. All auto-scaling limits raised                              | \$1,800 single day (w/ \$2800 during infra) | 10x traffic patterns, of downtime                     |
| Nov 28 - Dec 3 (wind-down)    | Scale in incrementally. Monitor that auto-scaling                             | Base line going naturally down              | Verify task scaling is traffic going back to baseline |

19-22

VPC Endpoints — Eliminate NAT Gateway Data Transfer Costs

ShopFlow was routing all AWS service traffic (S3, DynamoDB, SQS, Kinesis, ECR) through 3 NAT Gateways at \$0.045/GB. VPC Endpoints keep that traffic inside the AWS network — free for Gateway types, cheap for Interface types.

aws > VPC > Endpoints > Create endpoint

Create S3 Gateway Endpoint (FREE)

Gateway Endpoint — S3 (no charge for data transfer)

Service name

com.amazonaws.us-east-1.s3 (Gateway)

VPC

shopflow-prod-vpc (vpc-0abc123)

Route table: private-rt-1a

Route table: private-rt-1b

Route table: private-rt-1c

Gateway Endpoints for S3 and DynamoDB are FREE — no hourly charge and no data processing charge. All traffic to S3/DynamoDB from your VPC will automatically route through the endpoint. This eliminates NAT Gateway charges for S3/DynamoDB traffic (~\$340/month for ShopFlow).

Create endpoint

| VPC Endpoint             | Type           | Cost                  | ShopFlow Traffic Redirected                                                 | Monthly Savings         |
|--------------------------|----------------|-----------------------|-----------------------------------------------------------------------------|-------------------------|
| S3                       | Gateway — FREE | \$0                   | Firehose -> S3, application logs -> S3, product catalog, S3, AWS logs, etc. | \$340/month NAT avoided |
| DynamoDB                 | Gateway — FREE | \$0                   | Session service, cart service, product cache, etc.                          | \$180/month NAT avoided |
| ECR (container registry) | Interface      | \$0.01/hr + \$0.01/GB | ECS task pulls Docker images from ECR on prod                               | \$90/month NAT avoided  |
| SQS                      | Interface      | \$0.01/hr + \$0.01/GB | Order-svc -> SQS order queue, notification-svc                              | \$45/SQS NAT avoided    |
| Kinesis                  | Interface      | \$0.01/hr + \$0.01/GB | Spring Boot microservices -> shopflow-use-events                            | \$60/month NAT avoided  |

ShopFlow's ElastiCache cluster was provisioned with 3 x cache.r6g.xlarge nodes (12 GB total) based on a "generous estimate." Actual Redis memory utilization: 34%. That is 8 GB of RAM costing \$180/month sitting idle.

| Metric                  | Before Optimization                               | After Optimization                                    |
|-------------------------|---------------------------------------------------|-------------------------------------------------------|
| Node type               | cache.r6g.xlarge (4 vCPU, 12.93 GB RAM) x 3 nodes | cache.r6g.large (2 vCPU, 6.38 GB RAM) x 3 nodes       |
| Total cluster RAM       | 38.8 GB                                           | 19.1 GB                                               |
| Used RAM (p99)          | 13.2 GB (34% utilization)                         | 13.2 GB (69% utilization — healthy ceiling)           |
| Monthly cost            | \$980/month (on-demand)                           | \$490/month (on-demand) — then reserve 1 year for \$3 |
| Total after reservation | \$980/month                                       | \$336/month (-66%)                                    |

#### warn Never Drop Below 3 Nodes for Production Redis

Redis Cluster Mode requires minimum 3 shards for proper failover. Even after right-sizing node types, keep 3 nodes across 3 AZs. A single-node Redis is a single point of failure — one instance interruption takes down all caching, causing all requests to fall through to Aurora, which cannot handle the load.

Manual cost reviews happen when someone remembers. Automated cost reviews happen every week. ShopFlow built a Lambda that runs every Monday morning, compares costs to the prior week, and posts a Slack message with anomalies highlighted.

```
// Lambda: weekly cost digest posted to Slack (runs via EventBridge every Monday 9am ET) public
class WeeklyCostDigestLambda { private final CostExplorerClient ce =
CostExplorerClient.create(); private final SnsClient sns = SnsClient.create(); public void
handler(ScheduledEvent event) { LocalDate today = LocalDate.now(); LocalDate lastWeekStart =
today.minusDays(7); LocalDate priorWeekStart = today.minusDays(14); // Get cost by service for
last 7 days Map[String, Double] lastWeek = getCostByService(lastWeekStart, today); // Get cost
by service for prior 7 days Map[String, Double] priorWeek = getCostByService(priorWeekStart,
lastWeekStart); // Find services with > 15% increase AND > $100 absolute increase List[String]
anomalies = lastWeek.entrySet().stream().filter(e -> { double prior =
priorWeek.getOrDefault(e.getKey(), 0.0); double current = e.getValue(); double pctChange =
prior > 0 ? (current - prior) / prior * 100 : 100; return pctChange > 15 && (current - prior) >
100; }).map(e -> String.format("%s: $.0f (was $.0f, +%.1f%)", e.getKey(), e.getValue(),
priorWeek.getOrDefault(e.getKey(), 0.0), (e.getValue() - priorWeek.getOrDefault(e.getKey(),
0.0)) / priorWeek.getOrDefault(e.getKey(), 1.0) * 100)).collect(Collectors.toList()); // Post
summary to Slack via SNS -> Slack webhook String message = buildSlackMessage(lastWeek,
priorWeek, anomalies); sns.publish(r -> r.topicArn(COST_ALERTS_TOPIC).message(message)); } }
```

After applying the optimizations in this chapter, ShopFlow reduced its monthly AWS spend from \$47,000 to \$22,400 — a 52% reduction without any feature degradation or performance regression.

| Optimization                                                                | Monthly Before                       | Monthly After | Savings                              |
|-----------------------------------------------------------------------------|--------------------------------------|---------------|--------------------------------------|
| ECS Fargate right-sizing (all services)                                     | \$18,400                             | \$9,200       | \$9,200                              |
| Aurora Reserved Instances (1-year, no-upfront)                              | \$5,600                              | \$3,100       | \$2,500                              |
| Compute Savings Plans (70% baseline coverage)                               | Already counted in Fargate/EC2 lines | Applied       | ~\$2,800 additional discount applied |
| Data transfer (GraphQL sparse fields + VPC endpoints + same-AZ replication) | \$4,200                              | \$1,800       | \$2,400                              |
| ElastiCache right-sizing + 1-year Reserved Nodes                            | \$980                                | \$336         | \$644                                |
| S3 lifecycle policies (IA + Glacier tiering)                                | \$890                                | \$380         | \$510                                |

| Optimization                                                     | Monthly Before | Monthly After | Savings         |
|------------------------------------------------------------------|----------------|---------------|-----------------|
| Idle resource cleanup (dev clusters, orphaned ECRs, unused ALBs) | \$1,250        | \$230         | \$1,220         |
| Lambda power tuning (fraud detector, order processor)            | \$750          | \$489         | \$261           |
| DynamoDB on-demand -> provisioned (stable 1400s)                 | \$400          | \$190         | \$290           |
| CloudFront cache hit rate improvement (23% -> 33.4%)             | \$310          | \$185         | \$125           |
| TOTAL                                                            | \$47,000       | \$22,400      | \$24,600 (-52%) |

19-26

Reserved Instance Marketplace — Flexibility After Commitment

What if you over-committed on Reserved Instances and no longer need them? AWS has a secondary market where you can sell unused RIs to other AWS customers. ShopFlow used this when migrating from r5 to Graviton r6g instances.

| Scenario                                   | What Happened                                                                                                      | Resolution                                                                             |
|--------------------------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| ShopFlow migration to Graviton (r5 -> r6g) | Bought (1-year) RIs for r5.2xlarge Aurora instances. Didn't know Reserved Instance Marketplace (Graviton) existed. | Listed on Reserved Instance Marketplace. Sold (Graviton) for 20% less.                 |
| Dev environment shutdown                   | Bought t3.medium RIs for dev database (1-year term). Didn't know Reserved Instance Marketplace existed.            | Engineered t3.mediums with cheaply Allowed Reserved Instance Marketplace to sell them. |

tip The 1-Year No-Upfront Sweet Spot

ShopFlow always buys 1-year, No-Upfront Reserved Instances — never 3-year, and never All-Upfront. Reason: (1) AWS releases new instance generations every 12-18 months. Locking in for 3 years means missing Graviton improvements. (2) No-upfront preserves cash flow. (3) The discount difference between no-upfront and all-upfront is only 5-8% — not worth the cash commitment.

19-27

AWS Graviton — 20-40% Better Price-Performance

AWS Graviton processors (ARM-based, designed by AWS) deliver 20-40% better price-performance than equivalent Intel/AMD instances. ShopFlow migrated all ECS tasks and Aurora to Graviton — one of the highest ROI optimizations available.

| Component                             | Before (Intel x86)                                                                      | After (Graviton ARM)                                                                | Savings                                                | Migration Effort                        |
|---------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------|
| ECS Fargate tasks (all microservices) | x86_64 platform. Cost: \$9,200/month. 20% cheaper per 1.64U/month. Cost: \$7,360/month. | ARM64 platform. 20% cheaper per 1.64U/month. Cost: \$7,360/month.                   | \$1,840/month                                          | Low: no platform: linux/arm64 to x86_64 |
| Aurora PostgreSQL writer              | r5.2xlarge: \$0.48/hour                                                                 | r6g.2xlarge (Graviton): \$0.384/hour                                                | \$82.20/month                                          | Medium: snapshot + restore to r6g       |
| ElastiCache Redis                     | cache.r6g.large: already Graviton (cheap fix)                                           | Already on Graviton                                                                 | N/A — already optimized                                | None needed — r6g IS Graviton           |
| Lambda (where applicable)             | x86 architecture                                                                        | ARM64 (Graviton). 20% cheaper per 0.6 second Lambda share architecture set to arm64 | \$906.60/second Lambda share architecture set to arm64 | Low: no                                 |

19-28

Enforcing Tags with AWS Config — Catching Untagged Resources

A tagging strategy only works if tags are actually applied. ShopFlow uses AWS Config rules to detect untagged resources and auto-remediate by tagging them with "Team: unknown" and alerting the creator.

■ aws > AWS Config > Rules > Add rule

Required-Tags Config Rule

■ Managed Config Rule

Rule name (managed rule)  
required-tags ■

■ Rule Parameters

tag1Key  
Environment

tag1Value (optional — leave blank to require key with any value)

tag2Key  
Service

tag3Key  
Team

■ Remediation Action

Auto remediation  
AWS-TagResource (SSM Automation) ■

Auto-remediation parameter: TeamTag  
unknown

■ Resources evaluated: EC2, ECS, RDS, ElastiCache, Lambda, S3. Evaluation frequency: On configuration change. Non-compliant resources trigger SNS alert to DevOps Slack channel within 5 minutes.

Save rule

19-29

Cost Optimization Production Readiness Checklist

Before considering cost optimization "done," verify every item on this checklist. These are the 10 highest-impact actions for a growing startup on AWS.

| Priority    | Action                                                                                                            | Expected Savings                                                                            | Effort                                               |
|-------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------|
| 1 — Highest | Enable cost allocation tags on all resources (Environment, Service, Team) at 80% of baseline. Register resources. | Enabled, 80% of baseline. Each EC2 instance, Lambda function, S3 bucket, etc. tagged.       | 10 minutes per resource. Explore existing resources. |
| 2           | Set monthly budget in AWS Budgets with 80% and 95% alerts. Email or Slack alerts. Detection in 24 hours.          | Prevents bills from exceeding 80% of baseline. Alerts at 80% and 95%.                       | 15 minutes. Detection in 24 hours.                   |
| 3           | Analyze Compute Optimizer recommendations. Right-size EC2 instances. S3 Intelligent Tiering.                      | Right-size EC2 instances. S3 Intelligent Tiering. Up to 5% savings on EC2, up to 10% on S3. | 1 hour. (One-time setup, then monthly updates.)      |
| 4           | Buy Compute Savings Plans at 70% of baseline. 1-year commitment. Up to 70% savings.                               | Up to 70% savings on EC2. 1-year commitment. Up to 70% savings.                             | 1 hour. (One-time setup, then monthly updates.)      |
| 5           | Migrate ECS tasks and Lambda to Graviton ARM. 20% better price-performance.                                       | 20% better price-performance. 10% savings on ECS, 10% on Lambda.                            | 1 day. (One-time setup, then monthly updates.)       |
| 6           | Create VPC Gateway Endpoints for S3 and DynamoDB. Eliminate NAT Gateway data processing charges.                  | Eliminate NAT Gateway data processing charges. ShopFlow: \$520/month.                       | 1 hour. (One-time setup, then monthly updates.)      |
| 7           | Set S3 lifecycle policies: IA after 30 days, Glacier 90 days, delete after 1 year. S3 Intelligent Tiering.        | 50% savings on S3. 50% savings on S3. 50% savings on S3.                                    | 1 hour. (One-time setup, then monthly updates.)      |
| 8           | Reserve Aurora DB instances and ElastiCache nodes. 1-year commitment. Up to 75% savings.                          | Up to 75% savings on Aurora DB. Up to 75% savings on ElastiCache.                           | 1 hour. (One-time setup, then monthly updates.)      |
| 9           | Schedule auto-stop for non-production environments (Dev, Test). 15% reduction in staging environments.            | 15% reduction in staging environments. 15% reduction in staging environments.               | 1 hour. (One-time setup, then monthly updates.)      |

| Priority | Action                                                                                      | Expected Savings | Effort     |
|----------|---------------------------------------------------------------------------------------------|------------------|------------|
| 10       | Audit and delete idle resources monthly (orphaned EIPs, unused ASes, unattached S3 buckets) | \$1.2M/month     | 20 minutes |

19-30

Cost Optimization Anti-Patterns — What NOT to Do

ShopFlow's engineering team has made these mistakes. Learning from them is faster than making them yourself.

| Anti-Pattern                      | What Happened at ShopFlow                                                                                                | Consequence                                                                                                                          | Correct Approach                                                                                                |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Over-committing on Resources      | Bought 16 years of All-Upfront RIs for r5 instances. It took 18 months later to reach the goal of saving 20% on compute. | Lacked 18 months later, reached the goal of saving 20% on compute.                                                                   | Use On-Demand instances for short-term spikes and Reserved Instances for steady-state workloads.                |
| Optimizing before measuring       | Engineering team "optimized" the search service by reducing the number of shards and reducing the number of replicas.    | Search service performance degraded by 30% and the team had to spend \$500k on additional hardware to maintain the same performance. | Measure performance before optimizing. Use the AWS CloudWatch console to monitor search service performance.    |
| Removing monitoring to save costs | Removed CloudWatch Insights to save \$100k/month. Debugging was a nightmare.                                             | Lost \$100k/month in debugging. Debugging was a nightmare.                                                                           | Use CloudWatch Insights for monitoring. Use CloudWatch Logs for debugging.                                      |
| Ignoring data transfer costs      | Team focused entirely on compute costs. Data transfer costs were ignored.                                                | Data transfer costs grew to \$40k/month. Total cost of ownership increased by 10%.                                                   | Use Amazon S3 for data storage. Use Amazon ElastiCache for caching. Use Amazon CloudFront for content delivery. |

19-31

Review Q&A

| Q                                                                             | A                                                                                                                      |
|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| What is the most impactful first cost optimization step?                      | Tagging resources for cost allocation and a Cost Explorer review habit, you cannot identify what to optimize.          |
| How do you decide between on-demand and Savings Plans, or Reserved Instances? | On-demand for unpredictable spiky traffic (DynamoDB on-demand mode). Savings Plans for predictable traffic (EC2, RDS). |
| What is the difference between Graviton and Sparrow?                          | Graviton is a lower price performance for the same workload (20-40% cheaper). No interruptible instances.              |
| How does GraphQL or sparse fields affect data transfer costs?                 | REST APIs often transfer full objects: a product with 50 fields (5KB) even when the mobile client only needs 1 field.  |

19-32

AWS Cost Tools Quick Reference

A one-page reference for all the AWS cost management tools discussed in this chapter. Bookmark this and use it during your weekly Monday cost review.

| Tool                        | Where to Find It                                 | Best For                                                                     | ShopFlow Cadence                                                           |
|-----------------------------|--------------------------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| AWS Cost Explorer           | AWS Console -> Billing -> Cost Explorer          | Visualize spending trends by service, tag, or region.                        | Weekly (Mondays morning) review the Spend Explorer.                        |
| AWS Budgets                 | AWS Console -> Billing -> Budgets                | Set spend thresholds with email/SNS alerts.                                  | Set 10 budgets: 5 for departmental, 5 for global.                          |
| Cost and Usage Report (CUR) | AWS Console -> Billing -> Cost and Usage Report  | Most detailed billing data. Enables export to Athena or S3.                  | Automated every hour. Powers other daily reports.                          |
| AWS Compute Optimizer       | AWS Console -> Compute Optimizer                 | Machine learning recommendations for EC2, EKS, Fargate, Lambda, ElastiCache. | Monthly. Apply recommendations for EC2, EKS, Fargate, Lambda, ElastiCache. |
| Cost Anomaly Detection      | AWS Console -> Billing -> Cost Anomaly Detection | ML-based anomaly detection. Fires only for genuine spikes.                   | Automated. Alerts to finance Slack and PagerDuty.                          |
| Savings Plans               | AWS Console -> Billing -> Savings Plans          | Commit to a spend for 1-3 years to buy 5-66% discount.                       | Quarterly review. Buy for EC2, Fargate, Lambda.                            |

19-33

What's Next — Chapter 20 Preview

You have now mastered ShopFlow's cost optimization stack. In Chapter 20 (the final chapter), you will learn how all the building blocks come together into a High Availability and Disaster Recovery architecture — the design decisions that keep ShopFlow running when things go wrong.

| Chapter 20 Topic            | Preview                                                                                                                                      |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Multi-AZ Architecture       | How ShopFlow's ECS, Aurora, ElastiCache, and ALB are deployed across 3 Availability Zones — and why.                                         |
| Aurora Global Database      | How ShopFlow replicates Aurora across two AWS regions for < 1 second RPO and < 1 minute RTO.                                                 |
| Multi-Region Active-Passive | The DNS failover design using Route 53 health checks that routes all traffic to eu-west-1 within 60 seconds.                                 |
| Chaos Engineering           | How ShopFlow uses AWS Fault Injection Simulator (FIS) to intentionally terminate ECS tasks, inject API errors, and simulate network outages. |

| Chapter 20 Topic | Preview                                                                                                   |
|------------------|-----------------------------------------------------------------------------------------------------------|
| Recovery Runbook | The 10-step runbook ShopFlow's SRE team follows during a P1 incident: from first alert to post-incident r |